

Algorithmic Efficiency & Recursion Toolkit (AERT)

Unit-1 Assignment | Data Structures (ETCCDS202) | School of Engineering & Technology

Part A — Stack ADT

What is an ADT?

An Abstract Data Type (ADT) is a mathematical model for a data structure that defines the set of operations it supports and what those operations do, without specifying how they are implemented internally. The Stack ADT, for example, guarantees push, pop, and peek operations with LIFO behaviour — the caller does not need to know whether the underlying storage is an array or a linked list.

Why are Stack operations $O(1)$?

Push, pop, and peek all operate only on the top element of the stack. No traversal or shifting of other elements is required. Whether the stack holds 5 items or 5000, the number of steps for each operation stays constant — hence $O(1)$ time complexity. Space complexity for a single operation is also $O(1)$ since no auxiliary memory is needed.

The Stack ADT was used in Part C to store all Tower of Hanoi moves as they were generated, demonstrating that push and size() work correctly during a recursive run.

Part B — Factorial (Recursion)

The recursive factorial function reduces the problem by one at each step: $\text{factorial}(n) = n \times \text{factorial}(n-1)$, with $\text{factorial}(0) = 1$ as the base case. Negative inputs raise a `ValueError`.

n	factorial(n)
0	1
1	1
5	120
10	3,628,800

Complexity

Complexity	Notation	Explanation
Time – worst	$O(n)$	n recursive calls, each doing $O(1)$ work
Time – best	$\Omega(1)$	$n = 0$ or $n = 1$ returns immediately
Time – tight	$\Theta(n)$	always exactly n calls for any valid $n > 1$
Space	$O(n)$	call stack depth grows to n frames

Part B — Fibonacci (Naive vs Memoized)

Two versions were implemented and a call counter was added to each to measure how many recursive calls are made.

n	fib(n)	Naive calls	Memo calls
5	5	15	9
10	55	177	19

20	6765	21,891	39
30	832,040	2,692,537	59

Why is naive Fibonacci so slow?

In the naive version, $\text{fib}(n)$ calls $\text{fib}(n-1)$ and $\text{fib}(n-2)$, each of which spawns two more calls, and so on. The same sub-problems (e.g. $\text{fib}(18)$, $\text{fib}(17)$) are recomputed many times. The call tree has $O(2^n)$ nodes in the worst case, making the time complexity $O(2^n)$ — exponential growth. At $n=30$, over 2.6 million calls are made.

Memoization (Dynamic Programming)

Memoized Fibonacci stores each result in a dictionary the first time it is computed. Subsequent calls for the same n return the cached value immediately without recursing. Each unique sub-problem (0 to n) is solved exactly once, giving $O(n)$ time and $O(n)$ space. At $n=30$, only 59 calls are made compared to 2.6 million — a factor of ~45,000x improvement.

Version	Time	Space	Paradigm
<code>fib_naive</code>	$O(2^n)$	$O(n)$ stack depth	Pure recursion
<code>fib_memo</code>	$O(n)$	$O(n)$ memo dict	Dynamic Programming

Part C — Tower of Hanoi

The Tower of Hanoi problem asks us to move n disks from a source peg to a destination peg using an auxiliary peg, never placing a larger disk on a smaller one.

Move sequence for $N = 3$

1. Move disk 1 from A to C
2. Move disk 2 from A to B
3. Move disk 1 from C to B
4. Move disk 3 from A to C
5. Move disk 1 from B to A
6. Move disk 2 from B to C
7. Move disk 1 from A to C

Why $2^n - 1$ moves?

To move n disks: move the top $n-1$ disks to auxiliary ($T(n-1)$ moves), move the largest disk to destination (1 move), then move $n-1$ disks from auxiliary to destination ($T(n-1)$ moves again). This gives the recurrence $T(n) = 2 \cdot T(n-1) + 1$, with $T(1) = 1$. Solving: $T(n) = 2^n - 1$. For $N=3$: $2^3 - 1 = 7$ moves.

Complexity	Value	Explanation
Time	$O(2^n)$	Must make $2^n - 1$ moves; no way to do fewer
Space	$O(n)$	Maximum recursion depth is n stack frames

Part D — Recursive Binary Search

Binary search works on a sorted array by comparing the key to the middle element. If equal, return the index. If less, recurse on the left half; if greater, recurse on the right half. Return -1 if $\text{low} > \text{high}$ (not found).

Array	Search key	Result
[1,3,5,7,9,11,13]	7	index 3 (found)
[1,3,5,7,9,11,13]	1	index 0 (found)
[1,3,5,7,9,11,13]	13	index 6 (found)
[1,3,5,7,9,11,13]	2	-1 (not found)
[]	5	-1 (empty array)

Recurrence and Complexity

Each call halves the search space, giving the recurrence: $T(n) = T(n/2) + O(1)$. Unrolling: after k steps, $n/2^k = 1$, so $k = \log_2 n$. Time complexity is $O(\log n)$.

Case	Complexity	When it happens
Best	$O(1)$	Key is the middle element on the first call
Average	$O(\log n)$	Key found after roughly $\log n$ comparisons
Worst	$O(\log n)$	Key not present; search narrows to empty range

Part E — Algorithm Paradigm Reflection

Binary Search — Divide & Conquer

Binary search is a classic Divide & Conquer algorithm because at each step it divides the problem into two halves, discards one half based on a comparison, and conquers the remaining half recursively. The sub-problems are independent (no overlap) and smaller by a factor of 2 each time, which is the defining characteristic of Divide & Conquer. Real-life example: looking up a word in a printed dictionary by opening to the middle and deciding which half to continue in.

Fibonacci Memoized — Dynamic Programming

Naive Fibonacci has overlapping sub-problems: $\text{fib}(5)$ needs $\text{fib}(4)$ and $\text{fib}(3)$, but $\text{fib}(4)$ also needs $\text{fib}(3)$ — so $\text{fib}(3)$ is computed twice (and the overlap grows exponentially). Dynamic Programming solves this by storing the result of each sub-problem the first time it is computed (memoization) and reusing it on subsequent calls. This turns an exponential algorithm into a linear one. Real-life example: computing the shortest route in a GPS navigation system, where sub-routes are shared between many possible full routes.

Tower of Hanoi — Recursion (Decrease & Conquer)

Tower of Hanoi is solved by reducing the problem: to move n disks, first move $n-1$ disks out of the way (smaller sub-problem), move the largest disk directly, then move $n-1$ disks again. The base case is $n=1$: just move one disk directly. Each recursive call is for a strictly smaller n , guaranteeing termination. Unlike Divide & Conquer, the problem is not split into independent halves — the second recursive call depends on the outcome of the first.

Greedy — Conceptual Example

A Greedy algorithm makes the locally optimal choice at each step without reconsidering previous decisions. Example: the coin change problem with denominations [1, 5, 10, 25]. To make 41 cents, a greedy approach picks the largest coin that fits at each step: $25 + 10 + 5 + 1 = 41$ (4 coins). Greedy works here because coin denominations are structured so local optimality leads to a global optimum. However, greedy fails on arbitrary denominations — for denominations [1, 3, 4] and target 6, greedy gives $4+1+1$ (3 coins) but the optimal is $3+3$ (2 coins). That is why Dynamic Programming is needed for the general case.

References: Introduction to Algorithms (CLRS), 3rd Ed. — Cormen, Leiserson, Rivest, Stein. Python docs: `sys.setrecursionlimit()`. GeeksForGeeks — Tower of Hanoi recurrence derivation.